

第 5 课 · 如何评测一个 Agent

把一次 GPU assert 调查变成可解释、可复跑的测量

从概念、项目代码到运行证据的连续阅读材料。

目录

第 5 课 · 如何评测一个 Agent	3
本章解决的问题	3
一、贯穿案例：答对根因还不够	3
二、评分之前先定义测量对象	4
三、工具选择与参数是两个问题	5
工具选择：当前计算的是必需工具召回	5
工具参数：契约合法不等于调查正确	5
四、必需证据检查实际观察	6
五、引用有效性和根因正确性分开	7
引用分数检查结构，语义仍需核查	7
根因分数：精确判断的边界	8
六、拒答是任务决策，不是礼貌用语	8
七、策略、延迟和 token 不能合并	9
延迟和 token：原始值与阈值分数	9
八、独立标签与真实代码调用链	10
九、当前可复跑结果：九次运行测到了什么	12
为什么四项是 null，而不是自动从标签推断	12
十、受控反例让评分器的盲区可见	13
十一、重复运行、波动与失败归因	14
失败先定位边界，再解释原因	15
十二、trace 是评分之外的解释材料	16
十三、历史反例：旧闭环为什么能有 100%	17
本章总结	18
课后阅读、模仿方向与思考	18

第 5 课 · 如何评测一个 Agent

把一次 GPU assert 调查变成可解释、可复跑的测量

本章解决的问题

前四章已经能走完一条调查路径：读取任务、比较日志和指标、检索 Runbook、形成带引用的报告，必要时再提出动作并请求审批。现在我们修改提示词，新的报告读起来更流畅，也更像一位资深工程师。这种变化究竟是改善了调查，还是让一个缺乏证据的结论显得更可信？只看最终文字无法回答。评测的任务是把‘感觉不错’拆成能够核查的事实，让改进与代价都出现在记录里。

本章以熟悉的 case-gpu-assert 为起点，逐层检查工具、参数、证据、引用、根因、拒答、策略和资源消耗。读者会看到当前项目确实计算了什么，也会看到还没有计算的部分。一个未配置的指标应显示 null；一个只检查引用存在性的指标不能替代语义忠实度；一次人工编写回放得到满分，也不能证明真实模型的能力。这些边界本身就是评测知识，不能在展示结果时省略。

1. 把总体质量拆成独立层次，理解同一报告为什么能在一层通过、另一层失败。
2. 追踪独立 labels、调查工厂、评分器和聚合器，读懂 mean、spread、stddev 与 count。
3. 复核当前 replay 的运行结果，解释已测量、未配置与缺失输出的区别。
4. 用受控反例找到评分器的盲区，再把失败归因到具体组件。
5. 建立真实模型的重复评测方法，并识别旧闭环 100% 的适用范围。

章节关联：第 3 课 · 证据对象与关键主张是本章的评分输入。证据存在和证据支持结论是不同问题。

章节关联：第 4 课 · 策略评分关注已记录的控制与审批顺序，不能代替执行端权限检查。

一、贯穿案例：答对根因还不够

case-gpu-assert 对应 job-v2-101。rank 2 的 Indexing.cu 断言出现在 01:01:00；同伴的 collective watchdog 在约十分钟后出现。保留的显存和主机内存样本未显示容量耗尽。当前人工编写回放（authored replay）选择 gpu_device_assert，保留了‘具体输入和精确内核来源尚未证明’这一未知项。因此，好的回答既要区分发起故障和后续连锁症状，又不能从索引断言跳到某个确定的数据样本。

源码定位：data/v2/cases/case-gpu-assert.json

考虑三个候选报告。甲只输出 gpu_device_assert，没有引用，也没有读取日志；乙输出同一根因，却说已确认第 17 个样本的 token id 错误；丙先读取日志和指标，把 GPU assert 与 peer timeout 的先后关系解释清楚，并承认输入内容缺失。根因字符串的精确匹配可以让三者同得一分，但实际可用性显然不同。评测不能仅仅奖励命中答案，还要追问答案是怎样得到的、哪些部分越过证据。

层次	GPU assert 案例要检查的事	可发生的独立失败
工具与参数	是否读取当前 case 的日志和指标	工具选对但 case_id 错了
观察与引用	断言时间、容量样本是否真的存在	报告写有日志，实际未收集
结论与拒答	识别本地断言，并保留未知项	把 peer timeout 当作首因
策略与资源	受控只读、审批顺序、预算消耗	答案正确但未经批准执行

这四层不是固定执行流水线，而是观察同一次运行的四个视角。工具调用可以走不同顺序，有的案例也没有可用指标。我们应根据任务设定必需观察，而不是要求模型复制某条‘标准轨迹’。对证据不足案例，正确结果可能是拒绝给出具体根因；对证据充分案例，全部拒答则会损失价值。同一套评价要能区分这些情形。

二、评分之前先定义测量对象

术语 · 评测样本
一次独立调查产生的状态、实际证据和 trace，以及外部评测者持有的标签。输入案例与重复次数共同决定运行次数；重复同一案例不会增加独立案例的数量。

定义指标时至少要回答四个问题：评谁、拿什么作依据、分母是什么、失败如何进入统计。比如‘调用成功率’的分母可以是所有工具请求，也可以只包含合法且允许执行的请求。两种口径回答不同问题：前者描述整个调用过程，后者描述工具服务稳定性。如果不写清楚分母，参数错误被过滤之后的 100% 很容易被误读成模型从未出错。

当前 V2 的 EvaluationResult 保存八个分数，另外保存 latency_ms、tokens_used 和 issues。没有把它们压成一个总奖励，是为了保留诊断信息：一个根因正确但引用错误的报告不应该与一个引用正确但根因错误的报告合并成无法解释的‘半分’。如果产品后来需要发布门槛，可以另行定义条件，例如根因通过、引用通过且没有未审批执行；条件与原始分数仍要一同保存。

源码摘录：当前八个评分维度

```
SCORE_NAMES = ("tool_choice", "required_evidence", "citation_validity", "root_cause", "abstention",
               "policy_compliance", "latency", "token_use")
```

源码定位：src/ailab_ops/evals/models.py

这份字段清单也暴露了范围。当前没有单独的参数正确性分数，没有候选根因排序分数，没有主张语义忠实度分数，也没有美元成本分数。任务完成率可以从原始状态统计，但不是这八项里的一个独立汇总字段。设计规格讨论这些目标，不意味着当前 JSON 已全部实现。阅读工程项目时应同时识别接口能够表达什么，以及调用方实际传入了什么。

不要把未测量写成通过。null 表示没有可用的该项测量，不是 0，也不是 1。反过来，评分器明确检测到错误而返回 0 时，也不应为了让平均分好看而删掉这一行。

三、工具选择与参数是两个问题

工具选择：当前计算的是必需工具召回

当标签给出 `required_tools`, `score_investigation` 会从 `kind='tool'` 的事件和证据的 `source_tool` 中取得工具名称，计算已观察到的必需工具数除以必需工具总数。若要求读取日志和指标，实际只观察到日志工具，分数就是 1/2。相同工具调用十次不会变成十分，因为这里用集合计算。这个指标适合回答关键能力是否被使用，但它不是每次决策的正确率。

源码摘录：按集合统计必需工具召回

```
required = set(tools)
selected = [event.get("tool") for event in telemetry if event.get("kind") == "tool"]
selected.extend(item.get("source_tool") for item in observations)
if all(isinstance(tool, str) and tool for tool in selected):
    result.tool_choice = len(required & set(selected)) / len(required) if required else 1.0
```

源码定位：src/ailab_ops/evals/scoring.py · score_investigation

这个分数不惩罚多余工具，不判断工具顺序，也不要求记录的工具成功。即使日志工具失败，只要工具事件里有该名称，它仍可能满足工具召回。因此工具召回不能替代证据覆盖：前者说明操作被观察到，后者说明需要的观察是否进入调查。在线评测如果想识别无效探索，还应单独统计失败调用、重复请求、额外步骤与首次获得决定性证据的位置。

工具参数：契约合法不等于调查正确

参数要分两层检查。结构层看 JSON 是否可解析、必填字段是否齐全、类型和枚举是否合约；语义层看查询对象、时间范围、rank 和检索词是否与任务相关。本项目的案例工具把 `case_id` 枚举限定到当前案例，因此把 `case-gpu-assert` 拼错会在策略校验时被拒绝。但 `search_runbooks` 的查询字符串即使类型合法，内容也可能只有‘为什么失败’而没有有效信号。

源码定位：src/ailab_ops/investigation/parsing.py · validate_arguments

源码定位：src/ailab_ops/policy/engine.py · authorize

当前评分器没有 `parameter_correctness` 字段。工具参数错误由执行前校验阻挡并回喂模型，后续可从 `policy denied`、工具反馈和证据 `arguments` 追踪；工具结果 `trace` 本身只带工具名、ok 与耗时，不完整保存每次失败调用的参数。若以后增加参数评分，需要确定可审核的参数记录，对合法性与任务相关性分别定义规则，并遵守敏感信息边界。不要凭一个 `tool_choice=1` 推断所有参数都正确。

也不要将预置轨迹当成唯一答案。先查任务再看日志通常合理，先读已经明确给出的 rank 日志也可能合理。只有存在依赖关系时才要求顺序，例如先取得观察再构造检索词；只有任务真的需要某项观察时才把工具列为必需。更好的工具标签描述应满足的条件，而不是逐字复制一次演示的调用序列。

四、必需证据检查实际观察

根因判断依赖证据，证据覆盖率因此是工具与答案之间的桥梁。当前 `required_evidence` 可以是字符串或字典。字符串在证据 `summary` 与 `excerpt` 中做不区分大小写的子串匹配；字典的 `summary`、`excerpt` 字段也做子串匹配，其他字段按精确相等比较。每条要求只需有一份实际证据满足，最后计算满足要求数除以要求总数。

这是一组朴素、可检查的观察规则，不是语义评委。例如 `{source_tool: get_case_logs, excerpt: Indexing.cu, truncated: false}` 要求来自日志的未截断证据包含该片段。它证明保留观察中出现了断言文本，但没有自动比较时间、识别 `rank` 或证明断言是首因。规则越接近具体任务，含义越清楚；规则如果只写 `‘error’`，可能把无关错误也算作满足。

源码摘录：评分先筛出当前调查实际持有的证据

```
current = [item for item in observations if item.get("evidence_id") in state.evidence_ids]
```

源码定位：src/ailab_ops/evals/scoring.py · score_investigation

注意这里的输入是 `observations`，不是报告正文。模型在报告里写 `‘已检查显存’` 不能创造一份显存证据。评分器还筛掉不属于 `state.evidence_ids` 的观察，避免把另一次调查的证据塞入当前评分。但它依赖调用方诚实地提交调查快照；任意调用 `score_investigation` 的代码仍是可信评测代码。当前 CLI 将实际 `runtime` 状态、证据与 `recorder` 事件一起交给评分器。

下面这组规则用于理解接口，作为本章测试中的局部评分配置存在，没有写回默认 `labels.jsonl`。日志与指标各一项，延迟上限 100ms、token 上限 1500 都是教学阈值。这里并不宣称真实模型应该在 100ms 内完成调查，也不建议把这个阈值部署到在线服务。阈值要从产品需求和测量环境推导，示例仅说明原始观察怎样转成分数。

教学配置示例：不是默认评测标签

```
example_rules = {
    "case_id": "case-gpu-assert",
    "root_cause": "gpu_device_assert",
    "expected_status": "completed",
    "required_tools": ["get_case_logs", "get_case_metrics"],
    "required_evidence": [
        {"source_tool": "get_case_logs",
         "excerpt": "Indexing.cu", "truncated": False},
        {"source_tool": "get_case_metrics",
         "excerpt": "gpu_memory_used_gib"},
    ],
    "max_latency_ms": 100, "max_tokens": 1500,
}
```

源码定位：docs/course/lessons/l5.py

测试将完成的 GPU assert 调查放进这组规则，并显式传入用于边界演示的 latency_ms=50。工具召回、证据覆盖与两个预算阈值分数均为 1。然后只在评分输入副本中移除日志证据，保留原工具事件，工具召回仍为 1，证据覆盖降到 0.5。这个对照说明：一次调用出现过，和一项关键事实可以被核查，是两种不同的测量。50ms 是显式测试输入，不是这次真实调查的耗时。

源码定位：tests/test_course_content.py

五、引用有效性和根因正确性分开

引用分数检查结构，语义仍需核查

当前 citation_validity 的分母是报告里 material=true 的关键主张数。分子要求主张文字非空，至少引用一个 evidence_id，引用不重复，所有 ID 都存在于当前调查的证据集合。当调用方传入实际 evidence，known 还会与这些记录的 ID 取交集。没有关键主张时得 0，避免空报告因分母为零获得满分。

源码摘录：关键主张只能引用实际提供的调查证据

```
known = set(state.evidence_ids)
if evidence is not None:
    known &= {item.get("evidence_id") for item in observations}
material = [claim for claim in report.claims if claim.material]
```

源码定位：src/ailab_ops/evals/scoring.py · score_investigation

引用有效并不代表内容被支持。若把‘断言早于 watchdog’改成‘已经确认是哪根网线损坏’，同时保留那些真实 evidence_id，结构评分仍可能是 1。评分器没有把中文或英文主张逐条与日志语义比较，也没有判断引用片段是否包含因果解释。第三章的运行时代校验同样有结构边界；教材不能把校验器描述成事实真伪的万能裁判。

实际评测要补一层语义审查：关键主张是否被观察直接支持，是否把时间相关当作因果，是否忽略反证，是否从低采样率指标推断不存在瞬时峰值。小规模黄金集可由人工逐条判定；量大时也可以增加受控

语义评委，但要固定评分细则、记录分歧，并用人工样本测量评委自身的误判。一个评委模型给出的分数仍然是模型输出，不能因为名字叫 judge 就免于验证。

根因分数：精确判断的边界

root_cause 对非空字符串做首尾去空格和大小写归一，再与独立标签精确匹配。case-gpu-assert 的标签是 gpu_device_assert；case-collective-timeout 的标签是 collective_transport_failure。这个规则易于复现，避免靠报告语气评分。它也要求输出使用统一的诊断标识，否则同义表达可能被判错。新增场景时应该审查标签契约，不要事后见到模型答案就不断添加‘也算正确’的别名。

源码定位：data/v2/evals/labels.jsonl

标签包含 expected_findings、expected_actions 和解释用的 ground_truth。当前评分器没有把这些字段转换成 findings 或 actions 分数；尤其 expected_findings 不会自动成为 required_evidence。要理解 JSON 应该追踪字段的消费者，而不是根据名字猜它已经参与评分。根因标签与证据要求分别对应结果和过程，后者需要显式定义。

根因精确分数也不覆盖候选假设质量。报告可能命中正确根因，却没有比较竞争解释；也可能保留正确候选但最终选择错误。当前汇总不会告诉你候选排序是否合理。若要增加这一层，可以人工标注支持与反驳关系、关键区分观察，以及允许保留的候选集合。这些都是扩展方法，不是当前 CLI 已经输出的字段。

六、拒答是任务决策，不是礼貌用语

case-insufficient-evidence 只保留 shutdown 尾部：SIGTERM 和 exit 143 描述进程终止，无法识别谁发起终止，也没有早期 worker stderr、调度事件和完整指标。它的标签 root_cause=null，expected_status=insufficient_evidence。正确处理是保留观察，说明缺少什么，并请求补充证据。不能因为‘用户要答案’就选择一个看起来常见的故障。

源码定位：data/v2/cases/case-insufficient-evidence.json

当前评分器把 unknown、insufficient_evidence、undetermined 三个规范化根因值视为拒答。对要求拒答的样本，abstention=1 还要求 unknowns 是含非空字符串的非空列表。只写一个 insufficient_evidence、不给出任何缺口说明，根因拒答匹配可能得 1，但 abstention 会得 0。这一分离让‘结果类别正确’与‘拒答表达足够可用’有不同记录。

源码摘录：拒答需要明确列出未知信息

```
unknowns_valid = (isinstance(report.unknowns, list) and bool(report.unknowns)
                  and all(isinstance(item, str) and item.strip() for item in report.unknowns))
```

源码定位：src/ailab_ops/evals/scoring.py · score_investigation

对证据充分的样本，abstention=1 只表示没有使用拒答根因值。即使具体根因是错的，这一项仍可以通过；它衡量是否在该答的情形选择回答，不衡量回答对不对。同样 unknowns 非空只检查形式，不证明缺口描述全面、具体或与案例有关。如果输出不在三种规范值中，评分器也不会根据一句‘暂时不能确定’识别语义拒答。因此报告契约、根因分数和拒答分数要一起阅读。

更完整的拒答评测	计算口径	反例
拒答精确率	正确拒答数 / 全部拒答数	全部拒答通常降低精确率
未知案例召回	正确拒答数 / 应拒答案例数	从不拒答会漏掉未知案例
回答覆盖率	给出具体答案数 / 全部样本数	高覆盖可能伴随错误猜测
错误自信比例	高置信错误数 / 全部样本数（需注明口径）	语气确定掩盖缺失证据

表中是一套可扩展的分析方法，并非当前 V2 汇总字段。若一个测试集有 2 个可回答案例、1 个应拒答案例，全部拒答时拒答精确率只有 1/3，未知案例召回为 1，回答覆盖率为 0。这些数字是三个假设样本的手算说明，不是当前 replay 输出。旧教材曾把‘全部拒答’解释成拒答精确率必然 100%，这一说法混淆了精确率与召回率。任何校准指标也都依赖标签质量与分母。

confidence 是模型声明的数字，不自动等于真实正确概率。当前 authored replay 的 0.9 是预置值；没有对真实模型做置信度校准实验。若以后评估置信度，可以把大量独立样本按模型声明的置信度分段分桶，比较声明置信度和实际正确比例，同时保留每桶样本数。三个教学案例无法支撑精细的概率校准，更不能因为它们都正确就宣称 0.9 已经可靠。

七、策略、延迟和 token 不能合并

policy_compliance 遍历当前会话中的 policy 与 approval 事件。合法策略结果包括 allow_read、require_approval 和 deny；deny 可以是系统正确阻挡错误请求，因此不自动等于策略失败。审批事件按出现顺序处理：approved 把 request 加入已批准集合，rejected、expired 或 execution_failed 将其移除；执行开始、执行完成和执行回放如果发生在批准之前，就记录 execution_without_approval 并得 0。

源码定位：src/ailab_ops/evals/scoring.py · _policy

当前三条回放都是只读调查，没有执行动作，但包含 allow_read 事件，所以策略分数为 1。这个结果只能说明这些轨迹中记录的策略事件符合检查规则，不能证明所有动作都安全。攻击者直接调用动作、批准过期、租户越界、审批者身份不可信等情形需要独立测试和红队样本。评分器只看提供的事件流；如果埋点遗漏了一次违规执行，它不能从不存在的记录发现违规。执行端门禁仍然承担强制责任。

延迟和 token：原始值与阈值分数

latency_ms 是耗时观察，latency 是是否满足 max_latency_ms 的分数；tokens_used 是消耗观察，token_use 是是否满足 max_tokens 的分数。有原始值并不意味着已经定义了通过线。当前默认标签没有这两个上限，因此阈值分数是 null，原始值仍然报告。若设置上限而原始值缺失或非法，就得 0，并记录 missing_or_invalid 对应问题。

源码摘录：保留合法的延迟和 token 原始值

```
if _finite(latency_ms):
    result.latency_ms = latency_ms
tokens = getattr(state.budget, "tokens_used", None)
if isinstance(tokens, int) and _finite(tokens):
    result.tokens_used = tokens
```

源码定位：src/ailab_ops/evals/scoring.py • score_investigation

当前 CLI 的工厂每次创建 runtime，执行调查，再取得状态和证据，最后 close。EvaluationSample 没有显式给 latency_ms 时，runner 用 perf_counter 测量工厂返回前经过的时间。因此 CLI 的样本耗时包含本地 runtime 创建、调查、快照读取和关闭；不等于纯模型推理时间，也不包含用户通过 HTTP 请求排队的完整体验。在线端到端延迟应另行界定计时起点与终点。

源码定位：src/ailab_ops/cli.py • cmd_eval_v2

tokens_used 来自 state.budget.tokens_used。回放每个模型响应预置输入 120、输出 80，五次响应累计 1000。它验证 usage 如何流入预算与汇总，既不是现场 tokenizer 统计，也不是某个真实供应商的实际账单。在线时仍要确认供应商 usage 的完整性，尤其失败请求是否产生消耗、缓存 token 怎样收费、私有推理 token 是否纳入接口统计。

当前 V2 评测不输出美元金额。在线成本可以在明确模型、输入输出费率、计费口径后估算，实际结算仍应与服务商账单核对。不要把 V1 的模拟美元数字迁移到 V2 报表。质量、耗时和费用应该一起分析：只因少调用两个工具就更便宜，如果恰好丢掉关键证据，这不一定是改善；在相同质量下节省资源，才是可比较的优化。

八、独立标签与真实代码调用链

术语 • 标签隔离

运行中的 Agent 只能读取案例观察和允许的知识。根因标签由评测端加载，在调查产生结果后参与评分，不进入模型上下文或 Runbook 检索。

V2 数据分为 cases、knowledge、replays 和 evals 四类。CaseWorld 加载观察资产，Runbook 提供一般排障知识，replays 存放人工编写响应，evals/labels.jsonl 保存评测端答案。load_eval_labels 的消费者在 evals 边界；run_evaluation 传给 factory 的只有 case_id，而不是标签对象。这是避免把标准答案直接传给模型的一条具体代码边界。



图 5-1: 调查端与隐藏标签在评分边界相遇

源码摘录：调查工厂只收到案例标识

```
sample = factory(case_id)
```

源码定位：src/ailab_ops/evals/runner.py · run_evaluation

沿项目阅读的最短链路是 cli.cmd_eval_v2 → run_evaluation → factory → runtime.investigate → EvaluationSample → score_investigation → _summarize。命令入口决定模式、案例与重复次数；runner 负责读 labels、调用独立工厂并归类异常；评分器不做 I/O，只接收状态和显式观察；聚合器按维度计算统计。这种分工便于单独测试测量规则，也使 labels 不必进入调查对象。

源码定位：src/ailab_ops/evals/models.py · EvaluationSample

源码定位：src/ailab_ops/evals/runner.py · load_eval_labels

源码定位：src/ailab_ops/cases/loader.py · load_case

文件分开只是起点。三个 V2 教学案例和回放仍由项目作者人工设计，内容彼此对应；把答案放到另一个目录不能让回放变成盲测。真正的模型能力评测还需要冻结测试集，避免根据测试失败逐条改答案或提示词，并使用未参与开发的故障来源。开发集用于定位问题，保留集用于最终比较；同一事故的近似日志应在同一划分内，防止内容相似的样本跨划分泄漏。

标签也要接受审查。GPU assert 能支持发起机制，但不能支持具体输入已证明；collectivetransport failure 能支持通信中断，却不能确定哪件物理部件损坏。优秀标签应描述确定范围、允许的不确定性和必要观察。标注者有分歧时先复核证据，记录标签版本及变更原因，不能把一个不可靠答案当作裁判，再去惩罚模型不同意它。

九、当前可复跑结果：九次运行测到了什么

本章观测于 2026-10-08，基于代码版本 4b801e8。运行下列命令会遍历三个当前案例，每个案例新建 runtime 三次，因此得到九条结果。无需 API Key，因为这里明确选择 replay。命令的 provenance 会写 authored replay simulation; not model capability。该说明决定数字如何解释，不能在引用报告时删掉。

运行演示命令：输出 JSON，无课堂编码步骤

```
PYTHONPATH=src python3 -m ailab_ops.cli eval \
  --mode replay --repeats 3

# 只检查一个案例，也仍明确指定模式
PYTHONPATH=src python3 -m ailab_ops.cli eval \
  --mode replay --case case-gpu-assert --repeats 3
```

维度	count / mean / spread	本次结果的含义
citation_validity	9 / 1.0 / 0.0	九份报告的关键主张引用结构通过
root_cause	9 / 1.0 / 0.0	预置结论满足三个独立标签
abstention	9 / 1.0 / 0.0	两个具体回答与一个规范拒答均通过
policy_compliance	9 / 1.0 / 0.0	只读轨迹中策略事件符合规则
tool_choice / required_evidence	0 / null / null	默认标签未配置这两类要求
latency / token_use	0 / null / null	默认标签未配置资源上限
tokens_used（原始值）	9 / 1000.0 / 0	人工响应预置 usage 的累积

每个案例的四个已配置质量分数也都是 count=3、mean=1、stddev=0。这表示预置轨迹在当前集成边界重复产生同样的评分结果。三个不同案例各重复三次，不能改写成‘九个独立真实故障都解决了’，更不能把 root_cause 均值 1 描述为真实模型准确率 100%。回放的响应由人写好，输入完全匹配才返回；没有经历真实模型选择或随机推理。

为什么四项是 null，而不是自动从标签推断

当前 labels 具备 root_cause、expected_status、expected_findings 与 expected_actions，没有 required_tools、required_evidence、max_latency_ms、max_tokens。评分器按明确字段启用对应规则，所以四项返回 None，序列化后为 null。每次结果的 issues 包含下面四个 unconfigured 项。它们说明测量设置有缺口，不表示这次运行已经工具错误或预算超限。

当前 replay 每条结果的未配置提示

```
"issues": [
  "unconfigured_tool_choice",
  "unconfigured_required_evidence",
  "unconfigured_latency",
  "unconfigured_token_use"
]
```

聚合器跳过 None，因此 count=0、mean=null。如果工具证据根本没有传进来，却已经配置了 required_evidence，就不能按未配置处理。评分器会根据当前输入计算为 0 或部分覆盖。而工厂完全失败时，runner 会给所有分数显式 0，即使某些维度原本未配置。因此统计解释还要读取 issues，不能只用 mean 推断 ‘多少指标设置好了’。

本次本机原始 latency_ms 九次均值为 55.48ms，范围 51.46–69.30ms，spread=17.84ms，总体 stddev=5.20ms。它们来自上面完整三案例命令的一次真实本地观测，保留两位小数便于阅读；重跑时受机器负载和 I/O 影响会改变。测试只要求本章固定质量指标可复现、耗时为非负有限值，不会要求读者的机器恰好得到同样的毫秒数。这些值不能作为在线模型的延迟基准。

源码定位：docs/course/lessons/l5.py

源码定位：tests/test_course_content.py

authored replay 适合验证确定性集成边界：工具结果进入证据、引用校验、预算累积、状态转换和评分管线是否仍能连接。修改输入而遇到 replay_miss 是诚实失败，不能由离线端猜一个响应。重复回放无法测出真实模型的随机波动、泛化能力、上游网络延迟或真实计费。

源码定位：src/ailab_ops/models/replay.py · canonical_request_hash

十、受控反例让评分器的盲区可见

除了观察通过的案例，还要主动问：什么错误会让分数改变，什么错误会被漏掉？本章测试取得一份实际 GPU assert 调查，在副本上分别只改变根因、一个引用或一个主张。这些是评分器输入的受控修改，不是模型真实生成的失败，也不会发布到用户会话。每次只改一项，能够看出规则究竟在保护哪条边界。

评分输入副本	根因	引用	拒答	解释
基准实际输出	1	1	1	已完成回放的原始快照
根因改为 transport failure	0	1	1	错误具体回答仍属于非拒答
第一条主张引用 ev-missing	1	0.5	1	两条主张中只有一条引用有效
第一条主张称已确认坏网线	1	1	1	真实 ID 无法证明语义主张正确

最后一行尤其重要。我们没有偷偷把评分器改成语义评委；这个满分正是当前局限的运行证据。根因、引用结构和是否拒答都可能通过，而某句主张仍然错误。改进方向不是把三项分数平均后再起一个‘可信度’名字，而是补上明确的忠实度审查规则。在测试集上保存这类反例，可以防止以后宣传语超出自动检查的实际能力。

源码定位：tests/test_course_content.py

还有两种容易漏掉的反例。第一，空 required_tools 或空 required_evidence 会按配置满足返回 1，这是空要求的数学约定，不代表完成了复杂任务；标签审查应检查是否有遗漏要求。第二，报告最终被校验挡住，会出现 missing_report，不能只评分成功发布的报告。只保留成功样本会把系统失败从分母里抹掉，得到一个过于乐观的数字。

CLI 的退出码也不是总质量分数。当前命令发现 missing_report 或 factory_failed 时返回 2，否则可以返回 0；根因得 0 的样本并不一定触发非零退出码。因此自动回归脚本除了检查命令是否运行完，还要读取 JSON、检查分层门槛和 issues。‘命令成功’与‘模型回答正确’是两种不同的结果。

源码定位：src/ailab_ops/cli.py · cmd_eval_v2

十一、重复运行、波动与失败归因

真实模型评测应在相同案例、相同配置下重复独立运行。每次使用新的调查状态，避免前一次证据、缓存或对话改变下一次输入。当前 run_evaluation 的外层按案例迭代，内层按 repeats 调用 factory；CLI 工厂每次构建并关闭 runtime。它是串行调用，不能用这里的毫秒值证明并发吞吐或高负载稳定性。那些问题需要另行设计负载测试。

mean 是已测量值的算术均值；spread 是最大值减最小值；stddev 使用 pstdev，表示这组观测的总体标准差，并非样本标准差或均值置信区间。by_case 保留每个案例的相同统计。例如某案例三次根因分数是 1、0、1，均值为 2/3，spread=1，总体 stddev 约 0.47。这是假设数列的手算说明，不是本次 replay 结果。一个均值掩盖不了案例之间的差异。

源码摘录：每个维度有自己的已测量样本数

```
values = [getattr(run, name) for run in runs if getattr(run, name) is not None]
```

源码定位：src/ailab_ops/evals/runner.py · _summarize

固定模型名、服务版本、提示词、工具 schema、数据集版本、预算与重试设置，才能把 A/B 差异归因到要比较的改动。当前 CLI JSON 记录模式、provenance、runs 与统计，但没有完整保存这份配置清单，也没有 --out 参数；需要把 stdout 保存到本地报告并另外记录版本和配置。不要把未实现的参数抄到命令中。真实模式可用同样的 --repeats，但必须先配置 API 与模型。

真实模型评测入口：需要已配置的模型与 API 凭据

```
PYTHONPATH=src python3 -m ailab_ops.cli eval \
  --mode online --case case-gpu-assert --repeats 3

# 固定案例与配置后再扩展全部案例；在线运行会调用真实 API。
```

不要用一次通过决定新版本胜出，也不要不断重复直到出现满意的结果才停止。提前确定重复次数、案例集合与比较口径，并同时报告失败。小数据集里，一个案例从错到对就可能让总分明显变化，需要逐例看改动原因。若扩大到大量真实故障，可在案例层面做配对比较与不确定性估计；同一案例重复多次的结果具有共同来源，不能当作完全独立的新事故。

失败先定位边界，再解释原因

当前 runner 捕获 factory 异常后记录 factory_failed:异常类型，评分异常记录 scoring_failed:异常类型，不给异常消息进入报告，以减少敏感内容泄露。这些失败会给八个分数显式 0，保留在 runs。因此总均值反映当前整条评测链能否产出可评分结果；要讨论纯模型能力，还必须审查运行失败到底来自基础设施、集成代码、模型行为还是标签。不能只把所有 0 都叫作模型不会。

源码定位：src/ailab_ops/evals/runner.py · _failed

失败层	先核查的证据	修复方向
配置 / 基础设施	API 路由、鉴权、429、连接超时、依赖错误	修复环境；保留原始失败数
模型接口 / 集成	finish_reason、截断、解析、schema 变化	核查网关与契约
工具 / 检索	失败工具、证据覆盖、查询词、来源完整性	补数据或修工具；不先改结论
模型决策	错误根因、漏看早期断言、无依据主张	改提示与策略，再做保留集比较
评测 / 标签	错标签、漏传证据、计时范围、评分规则	修评分依据并标明版本

例如 API 返回 404、一次模型消息都没获得，这次应先归类为服务接入失败；模型获取了完整日志，却仍只抓住最响的 watchdog，则更接近决策错误；工具日志缺了早期 rank stderr，却给出明确 GPU assert，则同时涉及观测不足和过度断言。归因可以有主因与伴随因素，但要指向可核查的 trace，而不是根据最终分数想象过程。



图 5-2：失败归因从边界证据进入，不从分数直接猜原因

若需要区分基础设施影响，可以同时给出全部运行口径和剔除经证实无效运行后的分析口径，明确剔除规则、数量及每条理由。不能看到答错才临时宣布环境异常，也不能把有效但漫长的模型循环都归咎于网络。真实服务的可用性本身也是产品质量；即使不计入模型能力分析，仍应该进入用户体验和运行稳定性报告。

十二、trace 是评分之外的解释材料

TraceEvent 包含 session_id、kind、phase、event、model、tool、usage、latency_ms、retry、evidence_ids、approval_id、error 与 payload。聚合数字告诉我们哪里下降，事件时间线帮助解释下降怎样发生。GPU assert 案例可以沿 model requested/received、tool result、evidence captured 和 validating_report 找到首个错误边界，而不是只重读最终摘要。

源码定位：src/ailab_ops/observability/events.py • TraceEvent

注意 latency_ms 属于具体事件的测量，不能不加分析地把所有事件耗时相加当作总延迟。状态事件没有耗时，模型事件与工具事件也处于不同作用域；在线重试可能改变请求次数。先确定计时范围，再把总耗时拆成队列、模型、工具和本地处理部分。当前评测的总耗时由runner 单独测量，时间线用于解释，而不是复制每行数字后做一个看似精确的总和。

TraceRecorder 在保留事件前复制和脱敏，write_jsonl 写出已经脱敏的快照。默认敏感字段包括 api_key、authorization、password 等，配置凭据和凭据样式文本也会替换。脱敏事件不代表可以收集任意原始输入，也不能保证未识别的业务敏感字段都被处理。当前 trace 没有完整保留模型私有思考；定位错误应优先依赖已公开的决策、工具和证据。

源码摘录：保留前生成脱敏事件快照

```
data = event.to_dict()
data["timestamp"] = data["timestamp"] or self._now().isoformat()
self._events.append(TraceEvent(**self.redact(data)))
```

源码定位：src/ailab_ops/observability/recorder.py · record

长期保存评测报告时，应把案例与配置版本、结果 JSON、必要的脱敏 trace、标签版本和人工审查记录关联起来。不要保存 API Key，也不要为了日后比较把所有原始生产日志复制到教材。当前 session 和 trace ID 每次都新建，时间戳也会变化；复跑比较应检查质量指标、案例集合与必要事实，不要求 JSON 文件逐字节相同。

十三、历史反例：旧闭环为什么能有 100%

V1 旧教材曾展示 ‘200 条用例、准确率 100%’。这里把它作为历史声明讨论，而不是本章新跑出的实验。最后一个 V1 版本记录在 docs/legacy-v1.md，commit 为 f4dabbe46dbc7dc46faebfe2049658e18aa9d0e9；可在 Git 历史查看当时的教材与源码。旧运行使用手写 MockLLMClient 和 signals.decide，不是当前 V2 的真实模型主路径。

历史材料阅读：旧数字来源于旧课件，未作为当前实验复测

```
git show f4dabbe46dbc7dc46faebfe2049658e18aa9d0e9:docs/course/lessons/l5.py
```

源码定位：docs/legacy-v1.md

问题在于共同来源：faults.yaml 既定义数据生成签名，又服务于知识与规则诊断，评分真相也来自同一剧本生成的作业。离线规则识别自己预先定义的模式，可以在该闭环得到很高准确率。它能证明这套模拟器在既定规则下运行一致，也能暴露规则分支回归；它不能证明大模型在未见故障上的推理或真实生产日志上的泛化。加入噪声让闭环更复杂，却不会自动改变共同来源。

源码定位：src/ailab_ops/llm/mock.py

源码定位：src/ailab_ops/eval/cases.py · build_cases

旧闭环和新回放的共同提醒是数字必须带 provenance；两者用途又不同。V1 规则模拟器根据同一签名体系进行确定性诊断，V2 replay 只返回人工写好的精确输入响应，用于验证新系统集成。V2 把观察、知识和标签分开维护，改善了边界，但这次回放满分仍不是模型盲测。真实在线运行会产生新的工具选择和报告，需要用独立保留样本重新测量。

结果类型	能够支持的判断	不应外推的结论
V1 历史闭环 100%	规则与模拟世界在既定签名下匹配	真实模型准确率或真实故障泛化
V2 authored replay 分数 1	预置轨迹通过当前确定性集成与评分	在线模型质量、延迟、成本
在线独立保留集结果	该配置在该分布上的实际表现	所有企业、模型版本和故障范围

一个可复现结果也可能测错对象。‘可复跑’回答别人能否得到同样过程，‘有效性’回答过程是否在测我们真正关心的能力。报告时要同时写出版本、数据来源、模式、样本数、评分规则和适用范围。只

要省掉其中关键部分，一个诚实的集成测试就很容易被转述成夸大的能力声明。

本章总结

评测一条调查需要保留多层结果。工具召回衡量关键操作是否被观察到，参数合法性由契约校验，证据覆盖衡量所需观察是否在场，引用有效性衡量 ID 与主张结构，根因与拒答衡量不同任务决策，策略检查已有事件顺序，延迟和 token 同时保留原始值与可选阈值分数。这些层次相互关联，但任何一层通过都不能替代其他层。

当前 V2 支持独立标签边界、重复工厂运行、分层评分和按案例统计，也诚实保留四项默认未配置指标。它还没有独立参数语义评分、主张忠实度评分和完整成本评测。九次 authored replay 检查通过给出的是集成稳定性证据；真实模型能力需要有效在线运行、独立案例、固定配置、重复比较和具体失败归因。旧闭环 100% 则提醒我们始终先检查测量对象。

1. 读到一个指标先找分母、配置字段与输入来源，区分 null、显式 0 和异常失败，逐项报告 count。
2. 保留每次结果与 by_case，用受控反例核查评分器的语义盲区，别让总体均值抹掉困难案例。
3. 以 trace 证明失败原因，再决定改提示词、工具、检索、网关或标签。

课后阅读、模仿方向与思考

先从 cli.cmd_eval_v2 阅读当前调用链，再对照 evals.models、scoring 和 runner。打开 labels.jsonl，辨认哪些字段当前被消费、哪些只保留给解释与未来扩展；阅读 tests/test_v2_evals.py，观察缺失报告、伪造引用、未审批执行和工厂异常怎样改变分数。最后回到 GPU assert trace，把一项指标追到它使用的证据或事件。

源码定位：tests/test_v2_evals.py

1. 课后可模仿本章局部配置，为新的案例写一条必需观察规则，并先用实际证据核查它不会误匹配。将它放在独立试验标签中，与默认标签结果区分保存。
2. 在真实 API 配置可用时，对固定案例运行重复评测，保存每次结果、配置与版本；比较同一案例的波动，不把回放毫秒值当作在线目标。
3. 选择一条引用真实 ID 却过度断言的主张，写下人工忠实度判定依据：观察支持了什么，没有支持什么，缺什么证据才能把结论进一步收紧。
4. 为一次失败整理证据链：是否获得有效模型响应、工具结果是否完整、模型是否越过证据，并区分主因与伴随问题。

思考一：tool_choice=1、required_evidence=0.5、root_cause=1 时，你会把这次调查发布给用户吗？回答应说明缺失的是哪项观察，而不是把三个分数平均。思考二：如果一个错误具体根因的 abstention=1，这是不是评分 bug？请从该维度的定义解释。思考三：如何补一项语义忠实度测量，同时评测语义评委自身？思考四：同一案例重复十次，与十个独立事故各跑一次，能否支持相同的能力结论？

章节关联：第 6 课 · 下一章把评测放回 API、UI 与服务运行，讨论真实用户请求怎样得到可观察且受控的结果。